Properties of the Discrete Fourier Transform

Exercise 2 numerical verification and recursive FFT report

This report checks the algebraic properties of the DFT on a reproducible real-valued test signal of length N=32. All calculations use the convention:

$$A_k = \sum_{j=0}^{N-1} a_j e^{-2\pi i k j / N}$$

Numerical validation table

Property	Maximum error	Tolerance	Status
Conjugation identity	1.269e-13	1e-10	PASS
Real-signal N-k symmetry	8.406e-14	1e-10	PASS
DFT / inverse DFT	1.061e-14	1e-10	PASS
Parseval theorem	7.105e-15	1e-10	PASS
Cooley--Tukey recombination	5.844e-14	1e-10	PASS
Recursive radix-2 FFT	3.972e-15	1e-10	PASS

Conjugation identity. For any signal, DFT(conjugate(a)) equals the conjugate of A at the reversed index (–k) modulo N.

Real-signal N-k symmetry. For real samples, A_k = conjugate(A_(N–k) mod N); therefore positive and negative frequency bins contain conjugate information.

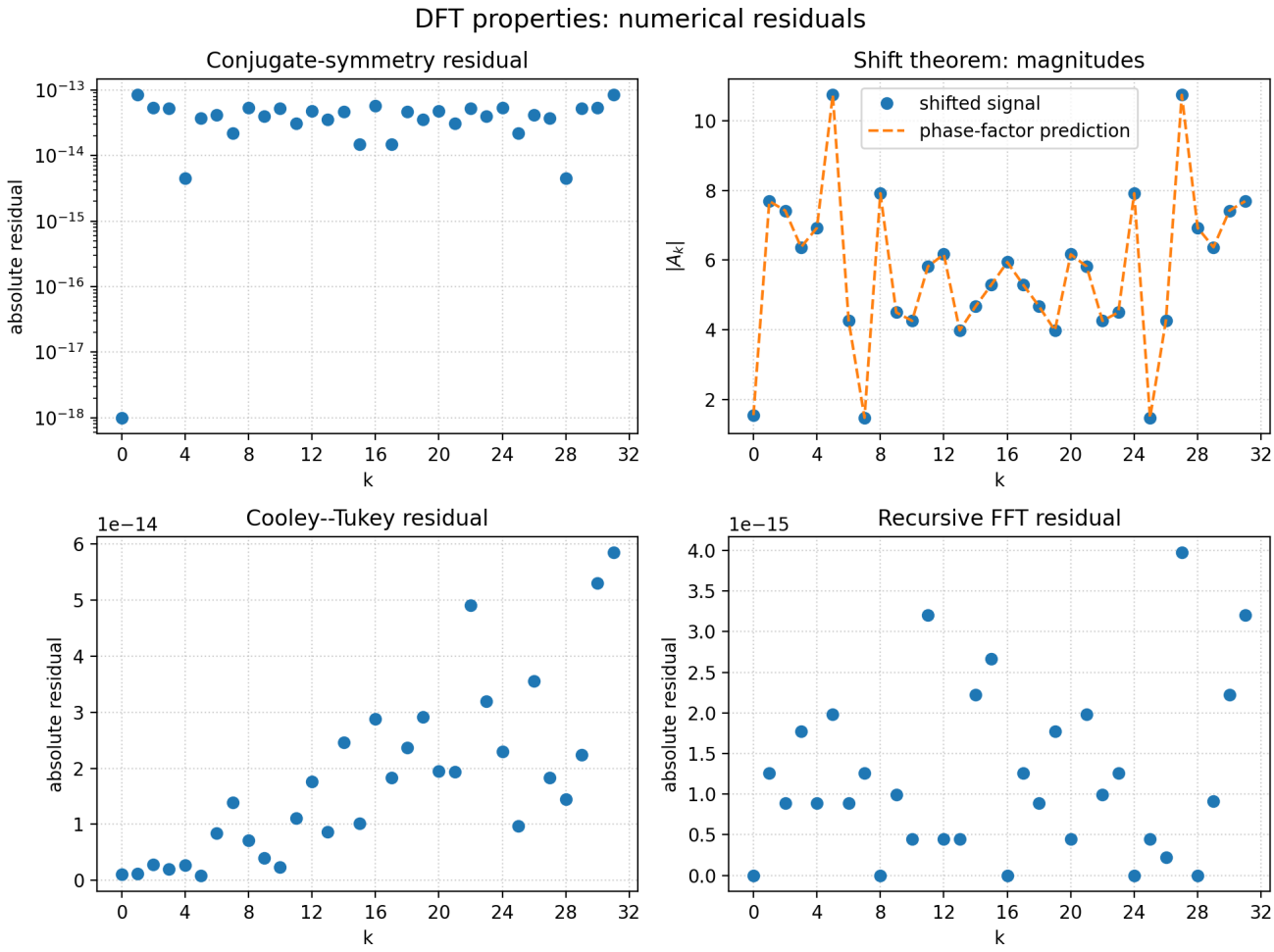
DFT / inverse DFT. Applying the inverse DFT to the DFT coefficients reconstructs the original signal sample by sample.

Parseval theorem. The sum of |a■|² equals 1/N times the sum of |A■|² for this DFT convention.

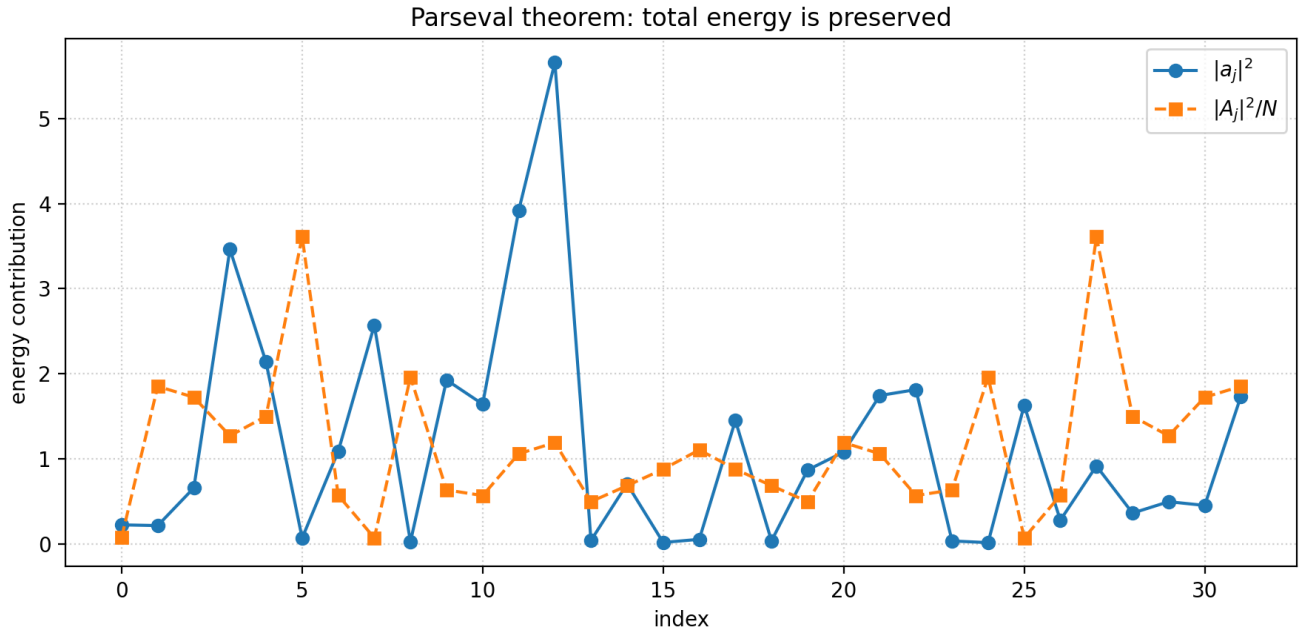
Cooley--Tukey recombination. The even and odd transforms combine as A■ = E■ + ω■■O■ and A■■■■ = E■ – ω■■O■.

Recursive radix-2 FFT. The recursive implementation agrees with NumPy's FFT.

Results and visual checks



The residuals are at floating-point round-off level. The shift theorem changes phase but not spectral magnitude, which is why the two magnitude curves overlap.



Parseval check: signal energy = 37.27627450, coefficient energy / N = 37.27627450.

Why are the triangle-wave coefficients purely real? The sampled triangle is real and even when it is centred at the origin. In the DFT, the terms at +j and –j then pair to a cosine, so the sine parts cancel and A_k is real. If the same triangle is shifted by m samples, the shift theorem multiplies A_k by exp(–2πikm/N), adding a phase. Thus the purely real coefficients are a consequence of centring/evenness, not of the triangle shape alone. The numerical shift-theorem residual for the test signal is 8.632e-14.

The DFT and its inverse

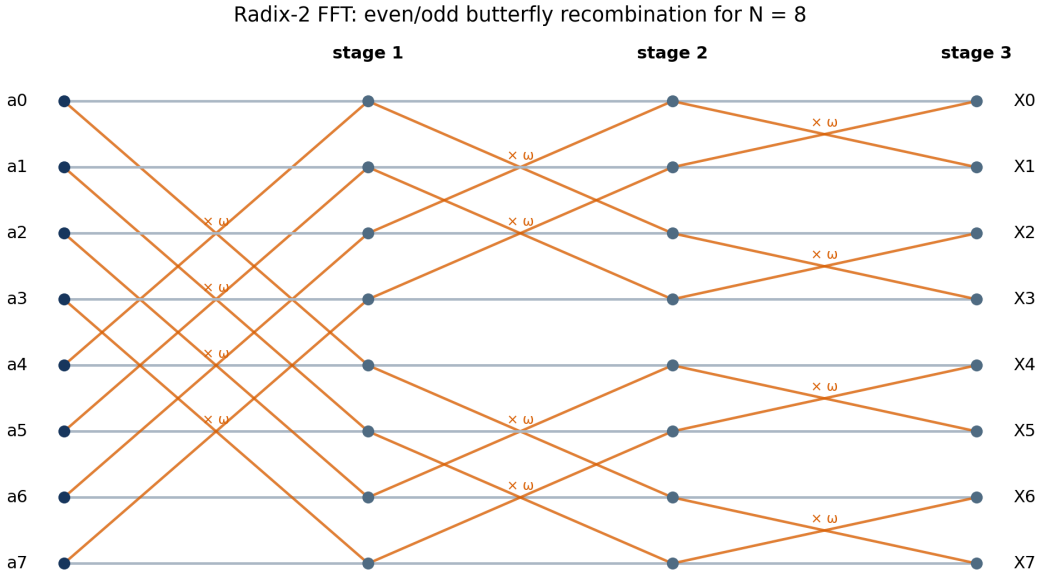
$$a_j = \frac{1}{N} \sum_{k=0}^{N-1} A_k e^{2\pi i k j / N}$$

The inverse contains the compensating factor 1/N. Numerically, the DFT/inverse-DFT round trip is included in the validation table above.

Cooley--Tukey recursion and radix-2 FFT

Splitting the samples into even and odd subsequences gives E■ = DFT(a■, a■, …) and O■ = DFT(a■, a■, …). With ω■ = exp(–2πi/N), applying the identities below recursively produces the radix-2 FFT, reducing the operation count from O(N²) to O(N log■ N).

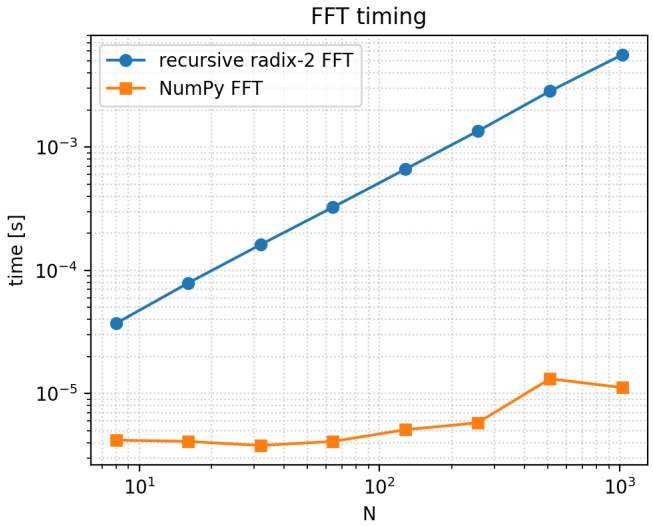
$$A_k = E_k + \omega_N^k O_k, \qquad A_{k+N/2} = E_k - \omega_N^k O_k$$

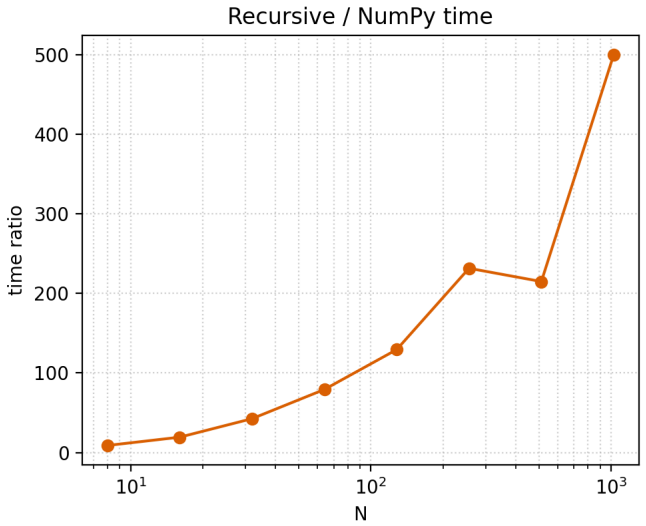


Each butterfly combines an even and an odd subtransform. The upper output uses E + ωO and the lower output uses E – ωO; applying this operation recursively is exactly what the code below implements.

```
def recursive_fft(values):
    if len(values) == 1:
        return values.copy()
    even = recursive_fft(values[::2])
    odd = recursive_fft(values[1::2])
    twiddle = exp(-2j*pi*arange(N//2)/N)
    upper = even + twiddle * odd
    lower = even - twiddle * odd
    return concatenate((upper, lower))
```

N	Recursive FFT [s]	NumPy FFT [s]	Ratio
8	3.710e-05	4.200e-06	8.83
16	7.900e-05	4.100e-06	19.27
32	1.616e-04	3.800e-06	42.53
64	3.250e-04	4.100e-06	79.27
128	6.601e-04	5.100e-06	129.43
256	1.344e-03	5.800e-06	231.68
512	2.838e-03	1.320e-05	215.01
1024	5.603e-03	1.120e-05	500.25


FFT timing
recursive radix-2 FFT
NumPy FFT
time [s]
10^{-3}
10^{-4}
10^{-5}
10^{1}
10^{2}
10^{3}
N


Recursive / NumPy time
time ratio
500
400
300
200
100
0
10^{1}
10^{2}
10^{3}
N